


```

layers.Dense(64, activation='relu'),
layers.Dense(10, activation='softmax')
])

```

Fully connected
Output layer

```

# Compile and train
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
model.fit(x_train, y_train, epochs=10, validation_data=(x_test, y_test))

```

Explanation:

Convolutional layers extract spatial features (e.g., edges, textures).

Max pooling reduces dimensionality.

Fully connected layers classify the features.

Applications: Image classification, object detection.

3. Recurrent Neural Networks (RNNs):

```

python
import tensorflow as tf
from tensorflow.keras import layers

```

Copy

```

# Generate dummy sequential data
import numpy as np
x_train = np.random.rand(1000, 10, 28) # 1000 sequences, each of length 10, with 28 features
y_train = np.random.randint(0, 10, 1000) # 1000 labels

```

```

# Build RNN model
model = tf.keras.Sequential([
    layers.SimpleRNN(64, return_sequences=True, input_shape=(10, 28)), # RNN layer
    layers.SimpleRNN(64), # Another RNN layer
    layers.Dense(10, activation='softmax') # Output layer
])

```

```

# Compile and train
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
model.fit(x_train, y_train, epochs=5)

```

Explanation:

The first RNN layer returns sequences for the next RNN layer.

The second RNN layer returns only the final output.

The output layer uses softmax for classification.

Applications: Time series forecasting, text generation.

4. Transformers:

```

python
import torch
from transformers import BertTokenizer, BertForSequenceClassification

```

Copy

```

# Load pre-trained BERT model and tokenizer
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')

```

```
model = BertForSequenceClassification.from_pretrained('bert-base-uncased')
```

```
# Tokenize input text
```

```
inputs = tokenizer("Hello, how are you?", return_tensors="pt")
```

```
# Forward pass
```

```
outputs = model(**inputs)
```

```
logits = outputs.logits
```

```
print(logits)
```

Explanation:

Tokenizers convert text into input IDs and attention masks.

The model outputs logits for classification.

Applications: Sentiment analysis, question answering.

5. Autoencoders:

```
python
```

```
import tensorflow as tf
```

```
from tensorflow.keras import layers
```

```
# Build autoencoder
```

```
encoder = tf.keras.Sequential([  
    layers.Dense(64, activation='relu', input_shape=(784,)),  
    layers.Dense(32, activation='relu')  
])
```

```
decoder = tf.keras.Sequential([  
    layers.Dense(64, activation='relu', input_shape=(32,)),  
    layers.Dense(784, activation='sigmoid')  
])
```

```
autoencoder = tf.keras.Sequential([encoder, decoder])  
autoencoder.compile(optimizer='adam', loss='mse')
```

```
# Train on MNIST
```

```
(x_train, _), (x_test, _) = tf.keras.datasets.mnist.load_data()  
x_train = x_train.reshape(-1, 784) / 255.0  
x_test = x_test.reshape(-1, 784) / 255.0
```

```
autoencoder.fit(x_train, x_train, epochs=10, validation_data=(x_test, x_test))
```

Explanation:

The encoder compresses input data into a lower-dimensional representation.

The decoder reconstructs the input from the compressed representation.

Applications: Dimensionality reduction, anomaly detection.

6. Generative Adversarial Networks (GANs):

```
python
```

Copy

Copy

```

import tensorflow as tf
from tensorflow.keras import layers

# Generator
generator = tf.keras.Sequential([
    layers.Dense(256, activation='relu', input_shape=(100,)),
    layers.Dense(512, activation='relu'),
    layers.Dense(784, activation='tanh')
])

# Discriminator
discriminator = tf.keras.Sequential([
    layers.Dense(512, activation='relu', input_shape=(784,)),
    layers.Dense(256, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])

# Combined GAN
gan = tf.keras.Sequential([generator, discriminator])
discriminator.compile(optimizer='adam', loss='binary_crossentropy')
gan.compile(optimizer='adam', loss='binary_crossentropy')

```

Explanation:

The generator creates fake data from random noise.

The discriminator distinguishes real from fake data.

Applications: Image generation, data augmentation.

7. Variational Autoencoders (VAEs):

```

python
import tensorflow as tf
from tensorflow.keras import layers

# Encoder
encoder = tf.keras.Sequential([
    layers.Dense(128, activation='relu', input_shape=(784,)),
    layers.Dense(64, activation='relu')
])

# Decoder
decoder = tf.keras.Sequential([
    layers.Dense(128, activation='relu', input_shape=(32,)),
    layers.Dense(784, activation='sigmoid')
])

# VAE
vae = tf.keras.Sequential([encoder, decoder])
vae.compile(optimizer='adam', loss='mse')

```

Copy

```
# Train on MNIST
(x_train, _), (x_test, _) = tf.keras.datasets.mnist.load_data()
x_train = x_train.reshape(-1, 784) / 255.0
x_test = x_test.reshape(-1, 784) / 255.0

vae.fit(x_train, x_train, epochs=10, validation_data=(x_test, x_test))
```

Explanation:

VAEs learn a probabilistic distribution of data.

The encoder outputs mean and variance for sampling.

Applications: Image generation, unsupervised learning.